



Season 2

Why coding the NES still rocks?

Programming the Nintendo Entertainment System (NES) still rocks because it sits at a rare sweet spot: simple enough to fully understand, yet powerful enough to make cool games.

Here are a few reasons why developers still love it:

1. You Can Understand the Entire Machine
2. Constraints Breed Creativity
3. Assembly Programming Is Fun
4. You Learn Real Computer Architecture
5. The Homebrew Scene Is Alive
6. It Feels Like Engineering
7. Every Success Feels Earned

Why coding the NES still rocks?

- A dedicated programmer can realistically learn how almost every part of the system works. That's incredibly satisfying.
- The combination of simplicity, constraints, craftsmanship, and deep hardware understanding is why NES programming remains one of the most enjoyable retro-development platforms today.

N. discontinues the product, the Community takes over

1. Nintendo stopped producing the NES decades ago. Official development stopped. Support ended.
2. Yet the platform is still alive because enthusiasts continue to create new games, build emulators, document technical details, preserve development tools and source code & repair original hardware.
3. Community Ownership: The best NES documentation today often comes from hobbyists, reverse engineers & homebrew developers

The system that once was a commercial product becomes a shared cultural project.

Masayuki Uemura (上村雅之) (1943 – 2021)

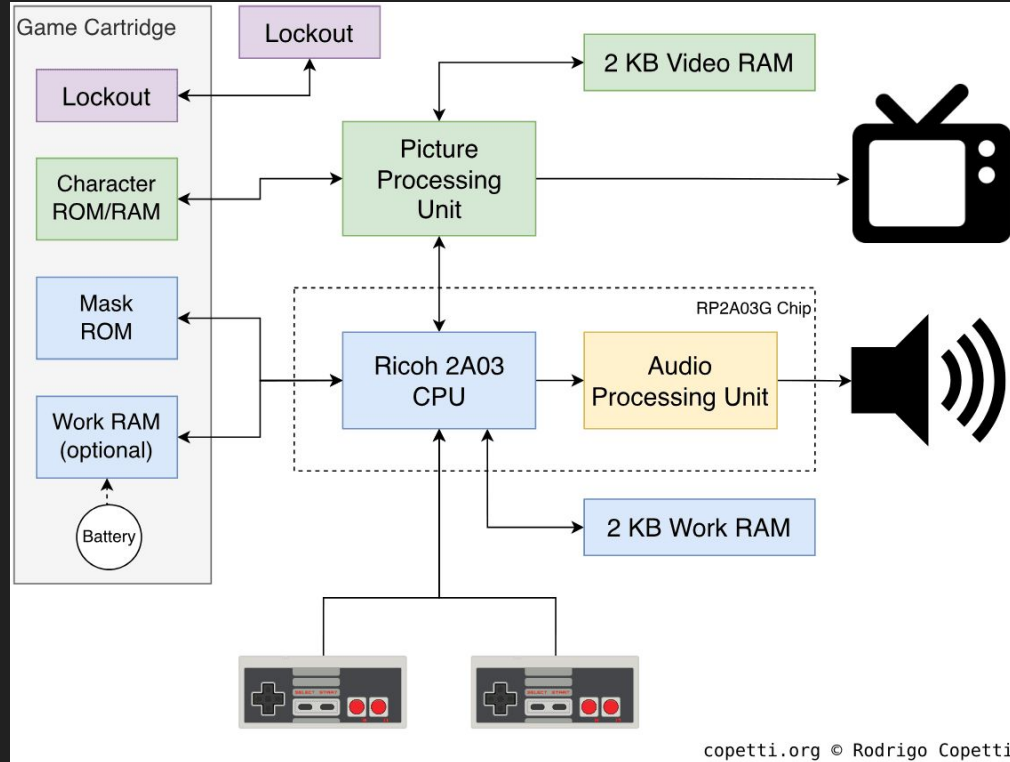


“Trabajando para Nintendo, me sentía como cuando era un niño y me sorprendía que todos esos hombres tan serios hiciesen juguetes”

- 1,79Mhz (NTSC)/1,66Mhz (PAL)
- 256x240 pixels
- 2KB de RAM y 2KB de VRAM
- 256 bytes for sprites
- 2 square channels, 1 triangle channel & 1 Noise channel



NES Architecture



Colors

\$00	\$01	\$02	\$03	\$04	\$05	\$06	\$07	\$08	\$09	\$0A	\$0B	\$0C	\$0F
\$10	\$11	\$12	\$13	\$14	\$15	\$16	\$17	\$18	\$19	\$1A	\$1B	\$1C	\$0F
\$20	\$21	\$22	\$23	\$24	\$25	\$26	\$27	\$28	\$29	\$2A	\$2B	\$2C	\$2D
\$30	\$31	\$32	\$33	\$34	\$35	\$36	\$37	\$38	\$39	\$3A	\$3B	\$3C	\$3D

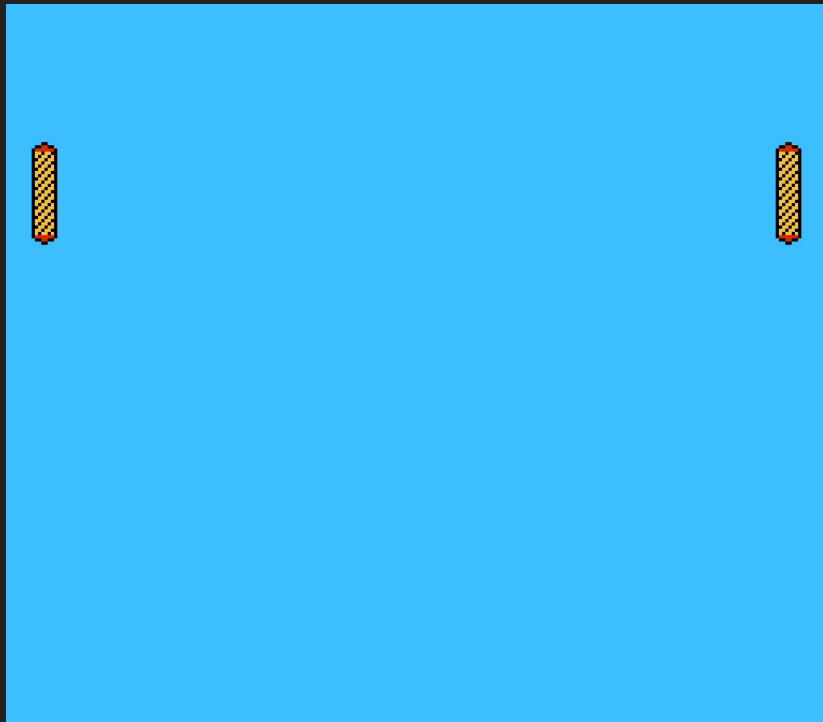
NES PPU

- 2 Backgrounds 256x240 px
- 64 Sprites. Max: 8 / scanline
- 2 x 256 Tiles. 8x8 px

Ms Patman Redux



PONG



Dev Tools

- Text Editor (Vscode, sublime, Vim,...)
- Assembler (XA65, CC65)
- NES Graphics Tools (NEXXT Studio, others)
- Sound Tracker (Furnace)

Homebrew cartridge

- Everdrive
- Flash Programmer





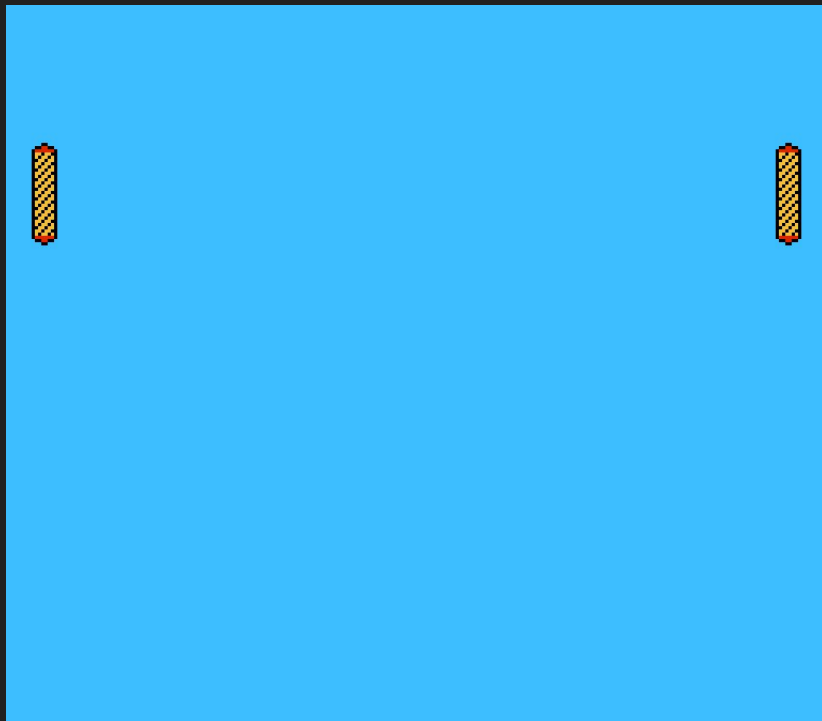
S2x02. Backgrounds

Backgrounds

- 2 screens of 256x240 pixels (vertical or
 - Memory Space For 4 Screens, Two of them mirroring the other one.
- Pixel Scroll (x,y)
- 4 Backgrounds Palettes
- Each 2x2 Square use the same Palette.
- Each Screen Has 32x30 tiles (960)
- Attribute table: 2 x (960+64 bytes)
 - 960 tile indexes
 - 64 palette bytes

Pong. Add a background

- Add a Background.
- Use 960 Tiles.
- Transform Picture to Tiles.
- Store Each Tile on CHR (ROM).
- Store Each Tile index on ROM.
- Send each Tile to VRAM.
- Draw Each Picture.



PNG? BMP? Tiles!!

Remember NES works With Tiles.

Each Image must be converted to Tiles and Store in the CHR (ROM) of the cartridge.

You can use a custom tool like:

- NEXXT Studio
- Other Tool (coming soon).
- JNesComp (By emiliollbb) (<https://github.com/emiliollbb/nescomp>)

Attribute Tables

- At the end of each nametable is a 64-byte region called an attribute table, which specifies which palette to use for each tile of background
- The palette of each square of 4x4 tiles is specified in a byte in the attribute table
- Each pair of 2 bits, specifies the palette for a square of 2x2 tiles

Attribute Tables

%01100011

Bits 0-1, top left:

%11 = palette 3

Bits 2-3, top right:

%00 = palette 0

Bits 4-5, bottom left:

%10 = palette 2

Bits 6-7, bottom right:

%01 = palette 1

